

手摸手实现分布式ID组件库

组件地址

```
<dependency>
  <groupId>org.opengoofy.index12306</groupId>
  <artifactId>index12306-distributedid-spring-boot-starter</artifactId>
  <version>${project.version}</version>
</dependency>
```

组件概述

1. 实现分布式唯一雪花算法 ID 生成器。
2. 封装分布式唯一雪花算法 ID 工具类。
3. 封装分库或分表基因算法工具类。

组件功能

雪花算法生成器

对雪花算法不了解先去查看相关概念，掌握后再继续查看本文档。

如何生成分布式雪花算法ID

首先，雪花算法生成器是为了解决工作机器 ID 重复问题，底层创建雪花算法依然采用推特那一套逻辑。

通过分配抢占的方式获取机器 ID，存储机器 ID 的位置就是个比较难选择的事情。

默认是通过 Redis 缓存存储，但是当项目中没有 Redis 时，采用随机数方式获取机器 ID。

- 1) 定义雪花算法获取机器 ID 模板抽象类。

采用模板方法模式，获取 Redis 或者随机数提供的机器 ID。

```

package org.opengoofy.index12306.framework.starter.distributedid.core.snowflake;

import cn.hutool.core.date.SystemClock;
import lombok.extern.slf4j.Slf4j;
import org.opengoofy.index12306.framework.starter.distributedid.toolkit.SnowflakeIdUtil;
import org.springframework.beans.factory.annotation.Value;

/**
 * 雪花算法模板生成
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Slf4j
public abstract class AbstractWorkIdChooseTemplate {

    /**
     * 是否使用 {@link SystemClock} 获取当前时间戳
     */
    @Value("${framework.distributed.id.snowflake.is-use-system-clock:false}")
    private boolean isUseSystemClock;

    /**
     * 根据自定义策略获取 WorkId 生成器
     *
     * @return
     */
    protected abstract WorkIdWrapper chooseWorkId();

    /**
     * 选择 WorkId 并初始化雪花
     */
    public void chooseAndInit() {
        // 模板方法模式：通过抽象方法获取 WorkId 包装器创建雪花算法
        WorkIdWrapper workIdWrapper = chooseWorkId();
        long workId = workIdWrapper.getWorkId();
        long dataCenterId = workIdWrapper.getDataCenterId();
        Snowflake snowflake = new Snowflake(workId, dataCenterId, isUseSystemClock);
        log.info("Snowflake type: {}, workId: {}, dataCenterId: {}",
this.getClass().getSimpleName(), workId, dataCenterId);
        SnowflakeIdUtil.initSnowflake(snowflake);
    }

```

```
}  
}
```

2) Redis 获取机器 ID 处理器。

```
package org.opengoofy.index12306.framework.starter.distributedid.core.snowflake;  
  
import cn.hutool.core.collection.CollUtil;  
import lombok.extern.slf4j.Slf4j;  
import org.opengoofy.index12306.framework.starter.bases.ApplicationContextHolder;  
import org.springframework.beans.factory.InitializingBean;  
import org.springframework.core.io.ClassPathResource;  
import org.springframework.data.redis.core.RedisTemplate;  
import org.springframework.data.redis.core.StringRedisTemplate;  
import org.springframework.data.redis.core.script.DefaultRedisScript;  
import org.springframework.scripting.support.ResourceScriptSource;  
  
import java.util.ArrayList;  
import java.util.List;  
  
/**  
 * 使用 Redis 获取雪花 WorkId  
 *  
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料  
 */  
@Slf4j  
public class LocalRedisWorkIdChoose extends AbstractWorkIdChooseTemplate implements  
    InitializingBean {  
  
    private RedisTemplate stringRedisTemplate;  
  
    public LocalRedisWorkIdChoose() {  
        this.stringRedisTemplate =  
            ApplicationContextHolder.getBean(StringRedisTemplate.class);  
    }  
  
    @Override  
    public WorkIdWrapper chooseWorkId() {  
        DefaultRedisScript redisScript = new DefaultRedisScript();  
        redisScript.setScriptSource(new ResourceScriptSource(new  
            ClassPathResource("lua/chooseWorkIdLua.lua")));  
    }  
}
```

```

        List<Long> luaResultList = null;
        try {
            redisScript.setResultType(List.class);
            luaResultList = (ArrayList) this.stringRedisTemplate.execute(redisScript,
null);
        } catch (Exception ex) {
            log.error("Redis Lua 脚本获取 WorkId 失败", ex);
        }
        return CollUtil.isNotEmpty(luaResultList) ? new
WorkIdWrapper(luaResultList.get(0), luaResultList.get(1)) : new
RandomWorkIdChoose().chooseWorkId();
    }

    @Override
    public void afterPropertiesSet() throws Exception {
        chooseAndInit();
    }
}

```

底层通过 Redis Lua 保障原子性。

脚本返回的 `resultWorkId`, `resultDataCenterId` 就是雪花算法组成本分的机器 ID 标识部分。

```

local hashKey = 'snowflake_work_id_key'
local dataCenterIdKey = 'dataCenterId'
local workIdKey = 'workId'

if (redis.call('exists', hashKey) == 0) then
    redis.call('hincrby', hashKey, dataCenterIdKey, 0)
    redis.call('hincrby', hashKey, workIdKey, 0)
    return { 0, 0 }
end

local dataCenterId = tonumber(redis.call('hget', hashKey, dataCenterIdKey))
local workId = tonumber(redis.call('hget', hashKey, workIdKey))

local max = 31
local resultWorkId = 0
local resultDataCenterId = 0

```

```

if (dataCenterId == max and workId == max) then
    redis.call('hset', hashKey, dataCenterIdKey, '0')
    redis.call('hset', hashKey, workIdKey, '0')

elseif (workId ~= max) then
    resultWorkId = redis.call('hincrby', hashKey, workIdKey, 1)
    resultDataCenterId = dataCenterId

elseif (dataCenterId ~= max) then
    resultWorkId = 0
    resultDataCenterId = redis.call('hincrby', hashKey, dataCenterIdKey, 1)
    redis.call('hset', hashKey, workIdKey, '0')

end

return { resultWorkId, resultDataCenterId }

```

如果说 Lua 脚本不容易理解，这里我将脚本翻译成 Java 执行结果如下：

```

public List<Long> translateLuaScript() {
    List<Long> result = new ArrayList<>();
    String hashKey = "snowflake_work_id_key";
    String dataCenterIdKey = "dataCenterId";
    String workIdKey = "workId";
    HashOperations<String, String, String> hashOps = redisTemplate.opsForHash();
    if (!redisTemplate.hasKey(hashKey)) {
        hashOps.increment(hashKey, dataCenterIdKey, 0L);
        hashOps.increment(hashKey, workIdKey, 0L);
        result.add(0L);
        result.add(0L);
        return result;
    }
    String dataCenterIdStr = hashOps.get(hashKey, dataCenterIdKey);
    String workIdStr = hashOps.get(hashKey, workIdKey);
    long dataCenterId = Long.parseLong(dataCenterIdStr != null ? dataCenterIdStr : "0");
    long workId = Long.parseLong(workIdStr != null ? workIdStr : "0");
    long max = 31;
    long resultWorkId = 0;
    long resultDataCenterId = 0;
    if (dataCenterId == max && workId == max) {

```

```

        hashOps.put(hashKey, dataCenterIdKey, "0");
        hashOps.put(hashKey, workIdKey, "0");
    } else if (workId != max) {
        resultWorkId = hashOps.increment(hashKey, workIdKey, 1L);
        resultDataCenterId = dataCenterId;
    } else if (dataCenterId != max) {
        resultWorkId = 0;
        resultDataCenterId = hashOps.increment(hashKey, dataCenterIdKey, 1L);
        hashOps.put(hashKey, workIdKey, "0");
    }
    result.add(resultDataCenterId);
    result.add(resultWorkId);
    return result;
}

```

3) 随机数生成机器 ID 处理器。

整体比较简单，调用 Random 随机函数获取两个值。就不过多介绍。

```

package org.opengoofy.index12306.framework.starter.distributedid.core.snowflake;

import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.InitializingBean;

/**
 * 使用随机数获取雪花 WorkId
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Slf4j
public class RandomWorkIdChoose extends AbstractWorkIdChooseTemplate implements
    InitializingBean {

    @Override
    protected WorkIdWrapper chooseWorkId() {
        int start = 0, end = 31;
        return new WorkIdWrapper(getRandom(start, end), getRandom(start, end));
    }

    @Override
    public void afterPropertiesSet() throws Exception {

```

```

        chooseAndInit();
    }

    private static long getRandom(int start, int end) {
        long random = (long) (Math.random() * (end - start + 1) + start);
        return random;
    }
}

```

雪花算法工具类

对象属性 `SNOWFLAKE` 是在 `AbstractWorkIdChooseTemplate` 获取到机器 ID 标识位后初始化的。

```

package org.opengoofy.index12306.framework.starter.distributedid.toolkit;

import org.opengoofy.index12306.framework.starter.distributedid.core.snowflake.Snowflake;
import
org.opengoofy.index12306.framework.starter.distributedid.core.snowflake.SnowflakeIdInfo;
import
org.opengoofy.index12306.framework.starter.distributedid.handler.IdGeneratorManager;

/**
 * 分布式雪花 ID 生成器
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
public final class SnowflakeIdUtil {

    /**
     * 雪花算法对象
     */
    private static Snowflake SNOWFLAKE;

    /**
     * 初始化雪花算法
     */
    public static void initSnowflake(Snowflake snowflake) {
        SnowflakeIdUtil.SNOWFLAKE = snowflake;
    }
}

```

```
/**
 * 获取雪花算法实例
 */
public static Snowflake getInstance() {
    return SNOWFLAKE;
}

/**
 * 获取雪花算法下一个 ID
 */
public static long nextId() {
    return SNOWFLAKE.nextId();
}

/**
 * 获取雪花算法下一个字符串类型 ID
 */
public static String nextIdStr() {
    return Long.toString(nextId());
}

/**
 * 解析雪花算法生成的 ID 为对象
 */
public static SnowflakeIdInfo parseSnowflakeId(String snowflakeId) {
    return SNOWFLAKE.parseSnowflakeId(Long.parseLong(snowflakeId));
}

/**
 * 解析雪花算法生成的 ID 为对象
 */
public static SnowflakeIdInfo parseSnowflakeId(long snowflakeId) {
    return SNOWFLAKE.parseSnowflakeId(snowflakeId);
}

/**
 * 根据 {@param serviceId} 生成雪花算法 ID
 */
public static long nextIdByService(String serviceId) {
    return
```



```

IdGeneratorManager.getDefaultServiceIdGenerator().nextId(Long.parseLong(serviceId));
    }

    /**
     * 根据 {@param serviceId} 生成字符串类型雪花算法 ID
     */
    public static String nextIdStrByService(String serviceId) {
        return
IdGeneratorManager.getDefaultServiceIdGenerator().nextIdStr(Long.parseLong(serviceId));
    }

    /**
     * 根据 {@param serviceId} 生成字符串类型雪花算法 ID
     */
    public static String nextIdStrByService(String resource, long serviceId) {
        return IdGeneratorManager.getIdGenerator(resource).nextIdStr(serviceId);
    }

    /**
     * 根据 {@param serviceId} 生成字符串类型雪花算法 ID
     */
    public static String nextIdStrByService(String resource, String serviceId) {
        return IdGeneratorManager.getIdGenerator(resource).nextIdStr(serviceId);
    }

    /**
     * 解析雪花算法生成的 ID 为对象
     */
    public static SnowflakeIdInfo parseSnowflakeServiceId(String snowflakeId) {
        return
IdGeneratorManager.getDefaultServiceIdGenerator().parseSnowflakeId(Long.parseLong(snowflakeId));
    }

    /**
     * 解析雪花算法生成的 ID 为对象
     */
    public static SnowflakeIdInfo parseSnowflakeServiceId(String resource, String
snowflakeId) {
        return
IdGeneratorManager.getIdGenerator(resource).parseSnowflakeId(Long.parseLong(snowflakeId))

```

```
;
    }
}
```

分库或分表基因算法

TODO

更新: 2023-07-17 17:01:30

原文: <<https://www.yuque.com/magestack/12306/lc2yb8gxtvfdt7rp>>